INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA SÃO PAULO	INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DE SÃO PAULO	CAMPUS São Paulo
Lógica de Programação 1		Aulas 01 a 03
Professor: Francisco Veríssimo Luciano E-mail: fvluciano@ifsp.edu.br		

1 - Aula 001 – Introdução a Lógica

1.1 – Noções Preliminares de Lógica e Construção de Algoritmos

1.1.1 – Lógica

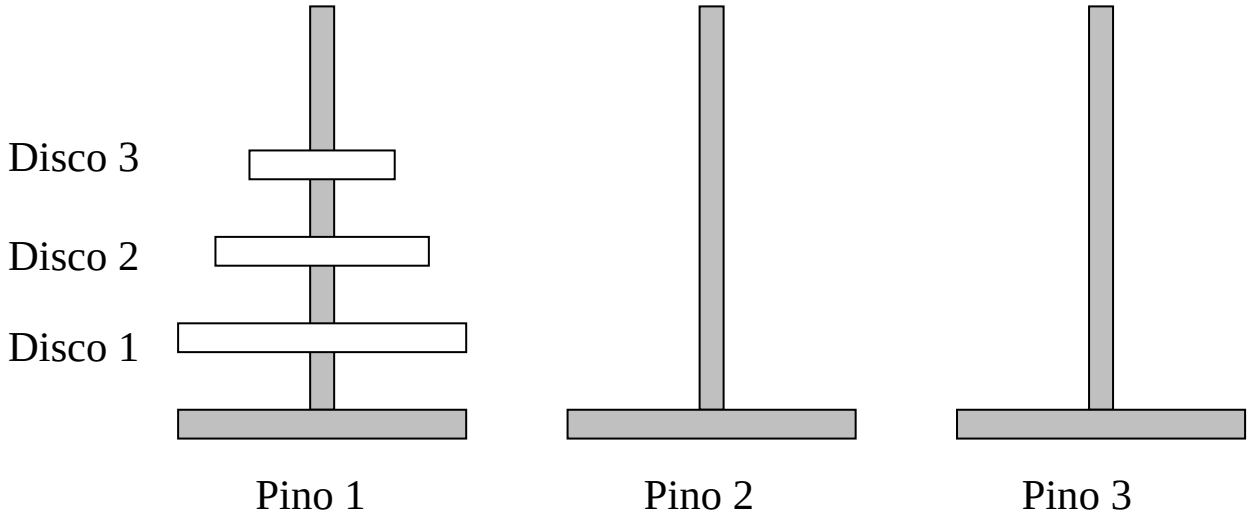
- Relacionada à capacidade de raciocinar
- Vinculada à capacidade de pensar
- Lógica de programação é a técnica de encadear pensamentos para atingir determinado objetivo.

1.1.2 – Algoritmo

- Um algoritmo é formalmente uma seqüência finita de passos que levam a execução de uma tarefa.
- Conjunto de instruções que definem um processo.

Um pequeno exemplo: Torre de Hanoi

- ✓ Descreva detalhadamente os procedimentos necessários para passar os 3 discos do Pino 1 para um dos outros Pinos, obedecendo às seguintes regras:
  - Mover somente um disco por vez;
  - Nunca poderá ficar um disco maior sobre um menor;



Canibais e Jesuítas

- ✓ Descreva passo a passo os procedimentos necessários para passar os 3 canibais e os 3 jesuítas da margem A para a margem B do rio, obedecendo às seguintes regras:
  - No barco só cabem duas pessoas;
  - Sempre terá que voltar alguém para buscar os demais;
  - O número de canibais nunca poderá ser superior ao de jesuítas em qualquer uma das margens (considerando-se quem está no barco também).

Algoritmos

Conceituando:
Algoritmo consiste em um conjunto lógico e finito de ações (instruções) que objetivam resolver um determinado problema.

Algoritmo não é **a** solução de um problema, pois, se assim fosse, cada problema teria um único algoritmo. Algoritmo é um dos caminhos para a solução de um problema e, em geral, os caminhos que levam a uma solução são muitos. **O aprendizado de algoritmos só é conseguido através de muitos exercícios.**

Não se aprende Algoritmos:
- Copiando Algoritmos
- Estudando Algoritmos

Só se aprende Algoritmos:
- Construindo Algoritmos
- Testando Algoritmos

Fases de um Algoritmo

Quando temos um problema e vamos utilizar um computador para resolvê-lo, inevitavelmente temos que passar pelas seguintes etapas:

a) Definir o problema;
b) Realizar um estudo de qual(is) a(s) forma(s) de resolver o problema;
c) Terminada a fase de estudo, elaborar um algoritmo contendo os passos lógicos a serem seguidos para a solução do problema;
d) Codificar os passos do algoritmo na linguagem de programação mais adequada;
e) Analisar junto aos usuários se o problema foi resolvido. Se a solução não foi encontrada, dever-ser-á retornar à fase de estudos a fim de localizar-se a falha.

Estas são, de forma bem geral, as etapas que um analista passa, desde a apresentação do problema até a sua efetiva solução. Iremos, neste curso, nos ater à etapa de estudo, também chamada de análise, e à etapa de programação. Mas, antes, vamos definir o seguinte conceito: programar um computador consiste em elaborar um conjunto finito de instruções, reconhecidas pelo mesmo, de forma que consiga executá-las. Estas instruções possuem regras e uma sintaxe próprias, como o português ou o inglês - sendo chamadas de **linguagem de programação**.

No mundo computacional existe uma grande variedade de linguagens: **Pascal, C, C++, Cobol, Fortran, Delphi, Visual Basic, Java,** etc.

Estrutura de Algoritmos

Antes de utilizarmos uma linguagem de computador, é necessário definir as ações a serem tomadas pela máquina de forma organizada e lógica, sem nos atermos às regras rígidas da sintaxe de uma linguagem. Para isso utilizaremos uma das formas de escrever tais algoritmos, conhecida como **Fluxograma** ou **Diagrama de Blocos**.

Formas de representação de um Algoritmo:

-**Fluxograma** (Diagrama de Blocos).
-**Portugol** (Português Estruturado ou Pseudocódigo);
-**Diagrama de Chapin;**

Exemplos:

Portugol, Português Estruturado ou Pseudocódigo;

```
algoritmo   teste
início
            A, B: inteiro
            A ← 1
            B ← 2
            se A > B então
                    A ← 5
            senão
                    A ← 10
            fim se
fim.
```

Fluxograma ou Diagrama de Blocos;

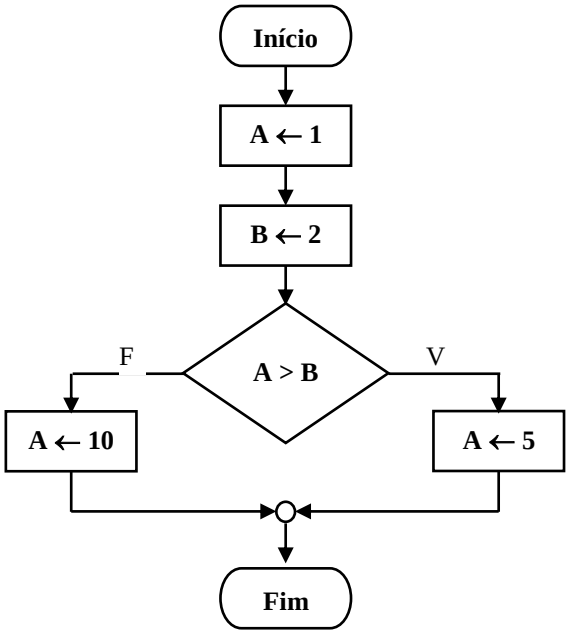
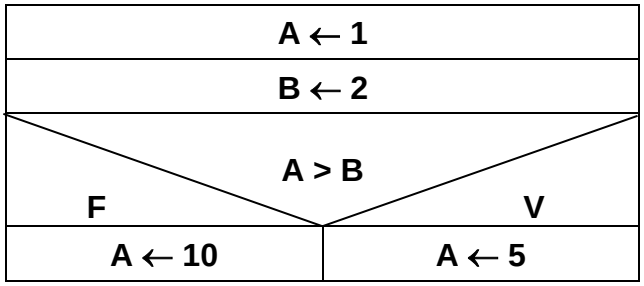


Diagrama de Chapin



Em Pascal, a solução apresentada pelos algoritmos acima ficaria da seguinte forma:

```
Program Teste;
 Var
    A , B : Integer ;
Begin
   A := 1 ;
   B := 2 ;
 if  ( A > B ) then
   A := 5
else
  A := 10 ;
End.
```

Variáveis

O computador possui uma área de armazenamento conhecida como memória. Todas as informações existentes no computador estão ou na memória primária (memória RAM – Random Access Memory - memória de acesso aleatório) ou na memória secundária (discos, fitas, CD-ROM, etc). Nós iremos trabalhar, neste curso, somente com a memória primária, especificamente com as informações armazenadas na RAM.

A memória do computador pode ser entendida como uma seqüência finita de caixas, que num dado momento, guardam algum tipo de informação, como um número, uma letra, uma palavra, uma frase, etc. Não importa ao usuário conhecer todas as informações contidas, basta apenas saber que lá sempre existe alguma informação.

O computador, para poder trabalhar como algumas destas informações, precisa saber onde, na memória, o dado está localizado. Fisicamente, cada caixa, ou cada posição de memória, possui um endereço, ou seja, um número que indica onde cada informação está localizada. Este número é representado através da notação hexadecimal, tendo o tamanho de quatro ou mais bytes. Abaixo seguem alguns exemplos:

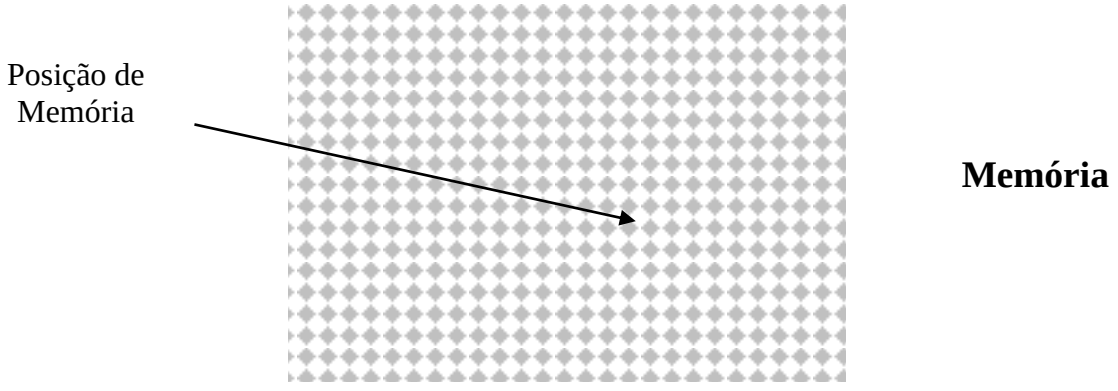
Endereço Físico	**Informação**
3000 : B712	‘João’
2000 : 12EC	12345
3000 : 0004	‘H’

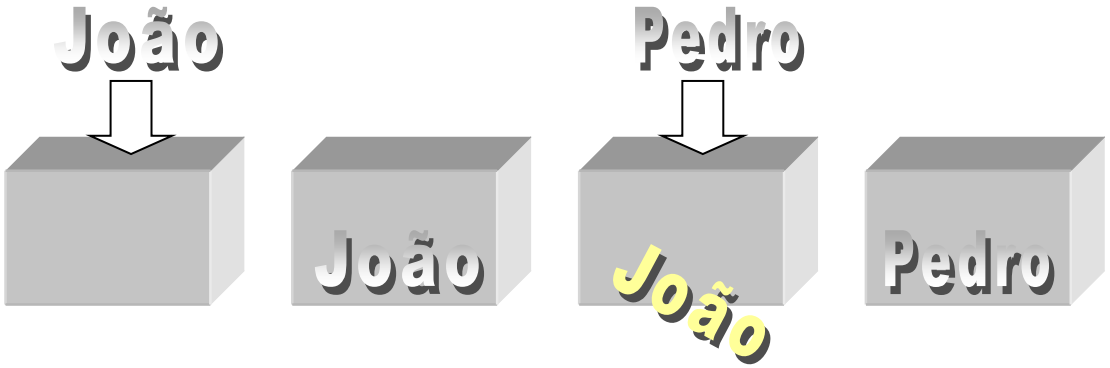
Como pode ser observado, o endereçamento das posições de memória através de números hexadecimais é perfeitamente compreendido pela máquina, mas para nós humanos torna-se uma tarefa complicada. Pensando nisto, as linguagens de programação de computadores facilitaram o manuseio das posições de memória da máquina, permitindo que, ao invés de trabalhar diretamente com os números hexadecimais, fosse possível <u>dar nomes</u> diferentes a cada posição de memória.

Os nomes são de livre escolha do usuário. Com este recurso, os usuários ficaram livres dos endereços físicos (números hexadecimais) e passaram a trabalhar com endereços lógicos (nomes dados pelos próprios usuários). Desta forma, o exemplo acima poderia ser alterado para ter o seguinte aspecto:

Endereço Físico	**Informação**
Nome	‘João’
Número	12345
Letra	‘H’

Como tínhamos falado, os endereços lógicos são como caixas, que num dado instante guardam algum tipo de informação. Mas é importante saber que o conteúdo desta caixa não é algo fixo ou permanente. Na verdade, uma caixa pode conter diversas informações, ou seja, como no exemplo acima, a caixa (Endereço Lógico) rotulada de “**Nome**” num dado momento contém a informação “**João**”, mas em um outro momento poderá conter uma outra informação, por exemplo “**Pedro**”. Com isto queremos dizer que o conteúdo destas caixas (endereços lógicos) **podem variar**, isto é, podem sofrer alterações em seu conteúdo. Tendo este conceito em mente, a partir de agora iremos chamar de forma genérica, as caixas ou endereços lógicos, de **variáveis**.





Conceituando:
Desta forma, é possível afirmar que uma variável é uma posição de memória, representada por um nome simbólico (atribuído pelo usuário), a qual contém, num dado instante, uma informação.

Nomes de Variáveis

- O nome de uma variável é formado por uma letra ou por uma letra seguida de letras ou números (quaisquer números);
- Não é permitido o uso de espaços em branco ou de qualquer outro caractere, que não seja letra ou número;
- Se utilizar mais de uma palavra para compor o nome da variável utilize o "_" (underline) para separar as palavras. Exemplo: data_nasc ;
- Não pode iniciar com um número.

Conteúdo de uma Variável

As variáveis podem armazenar:

- Valores numéricos;
- Textos (letras, palavras) ou até frases.

Identificadores: **Variáveis e Constantes**

Considere a fórmula matemática do cálculo do volume de uma esfera:

$$V = \frac{4}{3}\pi R^3$$

onde são encontrados valores que podem ser classificados como:

a) valores constantes, invariantes em todas as aplicações da fórmula, como são os casos dos valores 4, 3 e π $\rightarrow$ denominam-se **CONSTANTES**;

b) valores a serem substituídos na fórmula, em cada aplicação; a representação desses valores, usualmente é feita através de letras, que recebem o nome de **VARIÁVEIS** e tornam a fórmula genérica, possível de ser aplicada para resolver uma certa classe de problemas e não apenas um problema específico;

c) operações a serem feitas sobre determinados operandos (valores), para a obtenção da solução do problema.

Tipos de Dados

As Linguagens de Programação exigem que no momento que formos trabalhar com variáveis, **indiquemos o tipo de dado que a mesma poderá conter**, isto é, se uma dada posição de memória armazenará um **número** ou **uma letra,** etc.

A linguagem Pascal, por exemplo, possui a definição de alguns tipos que deverão ser usados para se trabalhar com variáveis. Esses tipos são:

Principais tipos de variáveis e dados que armazenam (relembrando...):

Algoritmo (***Portugol***)	**Pascal**	**Dados que podem armazenar**
a) Inteiro	a) INTEGER	Representa números entre –32.768 até +32.767. Ocupa 2 bytes na memória.
b) Real	b) REAL	Representa os números entre $2{,}9 \times 10^{-39}$ até $1{,}7 \times 10^{38}$. Ocupa 6 bytes na memória.
e) Caracter	e) CHAR	Representa um dos caracteres, da tabela ASCII. Ocupa 1 byte na memória.
d) Cadeia	f) STRING	Conjunto de caracteres (CHAR). Ocupa de 1 a 255 bytes na memória.
g) Lógica	g) BOOLEAN	Valor lógico. Assume somente dois valores: TRUE(Verdadeiro) ou FALSE(Falso). Ocupa 1 byte na memória.

Outros tipos de Variáveis

Pascal	Dados que podem armazenar
h) WORD	Números de 0 até 65.535. Ocupa 2 bytes na memória.
i) BYTE	Números de 0 até 255. Ocupa 1 byte na memória.
j) ShortInt	Representa os números entre -128 até 128 . Ocupa 1 bytes na memória.
l) LongInt	Representa os números entre - 2.147.483.648 até 2.147.483.648 . Ocupa 4 bytes na memória.
m) Single	Representa os números entre $1,5 \times 10^{-45}$ até $3,4 \times 10^{38}$. Ocupa 4 bytes na memória.
n) Double	Representa os números entre 5×10^{-324} até $1,7 \times 10^{308}$. Ocupa 8 bytes na memória.

Iremos gradativamente trabalhar com alguns destes tipos e mostrar as suas características.

Declaração de Variáveis

É necessário informar, logo no início do programa, quais as variáveis que serão utilizadas e que tipo de dados cada uma armazenará.

Para declarar uma ou mais variáveis, devemos utilizar, em Pascal, a palavra **VAR**. Nos algoritmos feitos em *Portugol* pode-se utilizar a mesma nomenclatura.

Nos algoritmos construídos com Fluxograma (Diagrama de Blocos) não é necessário declarar as variáveis. Apenas indicamos a atribuição de valores às variáveis necessárias (Fig. 1) ou a leitura dos dados para as mesmas(Fig. 2).

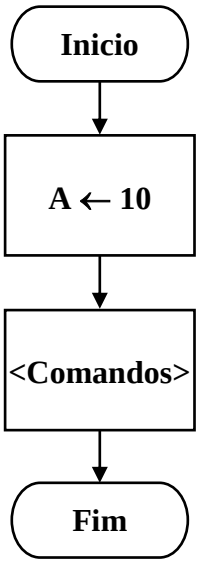


Figura 1

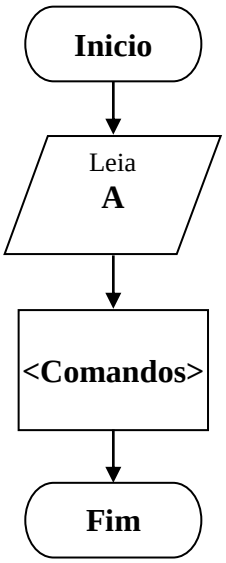


Figura 2

Abaixo seguem exemplos de algumas declarações de variáveis em Pascal:

a)
```
PROGRAM Teste ;
VAR
      A : INTEGER ;
BEGIN
      <comandos> ;
END .
```

b)
```
PROGRAM Teste ;
VAR
      Num : REAL ;
BEGIN
      <comandos> ;
END .
```

c)
```
PROGRAM Teste ;
VAR
      X, Palavra : STRING ;
BEGIN
      <comandos> ;
END .
```

d)
```
PROGRAM Teste ;
VAR
      B, Letra : CHAR ;
BEGIN
      <comandos> ;
END .
```

e)
```
PROGRAM Teste ;
VAR
      Y : REAL ;
      N : INTEGER ;
BEGIN
      <comandos> ;
END .
```

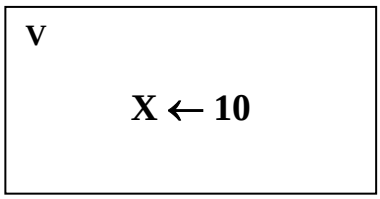
f)
```
PROGRAM Teste ;
VAR
      Letra, Caractere : CHAR ;
      Número : INTEGER ;
BEGIN
      <comandos> ;
END .
```

Obs: Os nomes dados às variáveis não podem ser os mesmos nomes de palavras reservadas do Pascal, tais como **PROGRAM, BEGIN, END, VAR**, etc…

Comandos de Atribuição

Quando definimos uma variável, é natural atribuirmos a ela uma informação. Uma das formas de colocar um valor dentro de uma variável, conseqüentemente colocando este dado na memória do computador, é através da **atribuição** direta do valor desejado à variável. Para isso são utilizados os símbolos **: =** (dois pontos igual) no Pascal e ← (seta para esquerda) em Algoritmos, que significam: **atribuir**, ou seja, a posição de memória que uma variável representa receberá uma informação, que será armazenada no interior desta variável. Exemplos:

Algoritmo



Pascal

```
PROGRAM Teste;
VAR
      X : INTEGER;
BEGIN
      X : = 10;
END.
```

O Exemplo anterior nos informa que:

a) Foi definida uma variável, a qual demos o Nome de " X ", e informamos que esta variável, ou posição de memória, só poderá aceitar dados que sejam numéricos e que estejam entre -32768 a +32767 (tipo INTEGER);

b) Atribuímos à variável " X " o valor 10.

A memória se comportaria da seguinte forma, de acordo com os itens acima:

Logo após a declaração da variável e antes da atribuição de valor:	
Variável	Conteúdo
X	Indefinido
Após a atribuição de valor:	
Variável	Conteúdo
X	10

Variáveis do tipo CHAR (Caractere) e STRING (Cadeia)

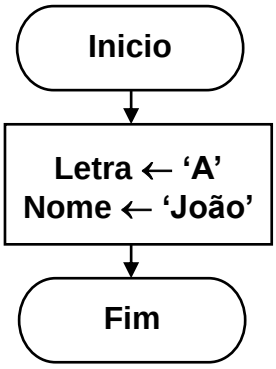
As definições de variáveis como sendo do tipo CHAR e STRING, possuem algumas curiosidades que merecem um cuidado especial por parte do usuário.

Uso do apóstrofro (')

Quando estivermos fazendo a atribuição de um valor para uma variável do tipo **CHAR** (Caractere) ou **STRING** (Cadeia), temos que ter o cuidado de colocar o valor (dado) entre apóstrofos (') - chamados por alguns de "aspas simples" - pois esta é a forma de dizer que a informação é caractere.

Exemplos:

Algoritmo



Pascal

```
PROGRAM AtribuiString;
VAR
    Letra : CHAR ;
    Nome : STRING ;
BEGIN
Letra := ' A ' ;
Nome := ' João ' ;
END.
```

Manipulação de caracteres individuais em STRING's (Cadeias)

Muitas vezes é necessário manipular caracteres individuais em uma STRING (Cadeia) . O Pascal possui uma forma toda especial de permitir tal operação, através do uso de colchetes([ ]) logo após o Nome da variável do tipo STRING (Cadeia) , e o número do caractere que se deseja manipular entre os colchetes.

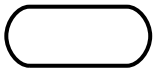
Formas de representação de algoritmos:

- ➔ Diagrama de Chapin (pronuncia-se “Chapã”)
- ➔ Diagrama de Blocos ou Fluxograma: representação gráfica do algoritmo
- ➔ Portugol, pseudocódigo ou português estruturado: descrição narrativa do algoritmo

Diagrama de Blocos ou Fluxograma

É um dos métodos utilizados para o desenvolvimento de algoritmos. Com esta técnica definimos os passos lógicos, a serem seguidos para a solução de um problema, com o auxílio de figuras geométricas. Posteriormente, codificarmos esses passos em uma linguagem de programação em um computador.

Há muitas figuras para construção de Diagramas de Blocos/Fluxogramas. As principais que utilizaremos, são:

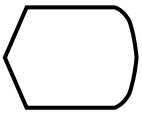


Terminal - para indicar *Início* ou *Fim* do programa;

Processamento - para cálculos, classificações, atribuições de valor, etc.;

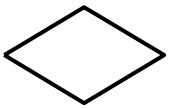
Entrada e Saída de Dados - é utilizado sempre em conjunto com os verbos *Leia*, para indicar a leitura de valores para variáveis, ou *Escreva*, para indicar a exibição de mensagens e/ou conteúdo de variáveis;

Entrada de Dados via teclado

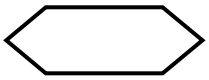


Saída ou Exibição de Dados em Vídeo - para indicar a exibição dos valores contidos nas variáveis e mensagens (textos) contidas nos programas;

Saída de Dados Impressos;



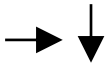
Decisão - para efetuar um desvio condicional, uma “pergunta”, para definir se o programa deve seguir numa direção ou em outra;



Estrutura de controle: repetição (laço para)

Conector - para ligar as seqüências de solução do problema vindas de diferentes pontos do Fluxograma;

Conector de Página - para ligar pontos do Fluxograma na mesma página ou em páginas diferentes; normalmente utiliza-se um número junto, para indicar de que ponto vem e para qual ponto deve seguir o fluxo;



Setas de Orientação do Fluxo - para indicar o sentido a ser seguido.

Operadores Aritméticos

+ (Adição)

- Tipo de operandos permitidos: inteiros, reais, cadeias de caracteres.
- Operação executada: no caso de inteiros e reais o operando à esquerda do **+** é somado ao operando a sua direita, sendo o tipo do resultado dessa operação dependente de seus operandos:
  - Se os dois operandos são inteiros, o resultado da soma é um valor inteiro;
  - Se os dois operandos são reais, o resultado da soma é um valor real;
  - Se um dos operandos é real e o outro é inteiro, o resultado da soma é um valor real;
  - No caso dos operandos serem ambos cadeias de caracteres o resultado da operação é dado pela cadeia obtida pela concatenação da cadeia dada pelo primeiro operando com a cadeia dada pelo segundo operando.

- (Subtração)

- Tipo de operandos permitidos: inteiros, reais.
- Operação executada: o operando à esquerda do **-** é subtraído do operando a sua direita, sendo o tipo do resultado dessa operação dependente de seus operandos:
  - Se os dois operandos são inteiros, o resultado da operação é inteiro;
  - Se os dois operandos são reais, o resultado da operação é real;
  - Se um dos operandos é real e o outro é inteiro, o resultado da operação é real.

*** (Multiplicação)**

- Tipo de operandos permitidos: inteiros, reais.
- Operação executada: o operando à esquerda do ***** é multiplicado pelo operando a sua direita, sendo o tipo do resultado dessa operação dependente de seus operandos:
  - Se os dois operandos são inteiros, o resultado da operação é um valor inteiro;
  - Se os dois operandos são reais, o resultado da operação é um valor real;
  - Se um dos operandos é real e o outro é inteiro, o resultado da operação é um valor real.

/ (Divisão)

- Tipo de operandos permitidos: inteiros, reais.
- Operação executada: o operando à esquerda do **/** é dividido pelo operando a sua direita, sendo o resultado dessa operação **real**.

DIV

- Tipo de operandos permitidos: ambos do tipo inteiro.
- Operação executada: o operando à esquerda do **DIV** é dividido pelo operando à sua direita, sendo o resultado desta operação um valor inteiro resultante da divisão.
- Exemplo: 7 DIV 3 resulta 2 (vide abaixo).

MOD

- Tipo de operandos permitidos: ambos do tipo inteiro.
- Operação executada: o operando à esquerda do **MOD** é dividido pelo operando à sua direita, sendo o resultado desta operação o resto inteiro da divisão.
- Exemplo: 7 MOD 3 resulta 1 (vide abaixo).

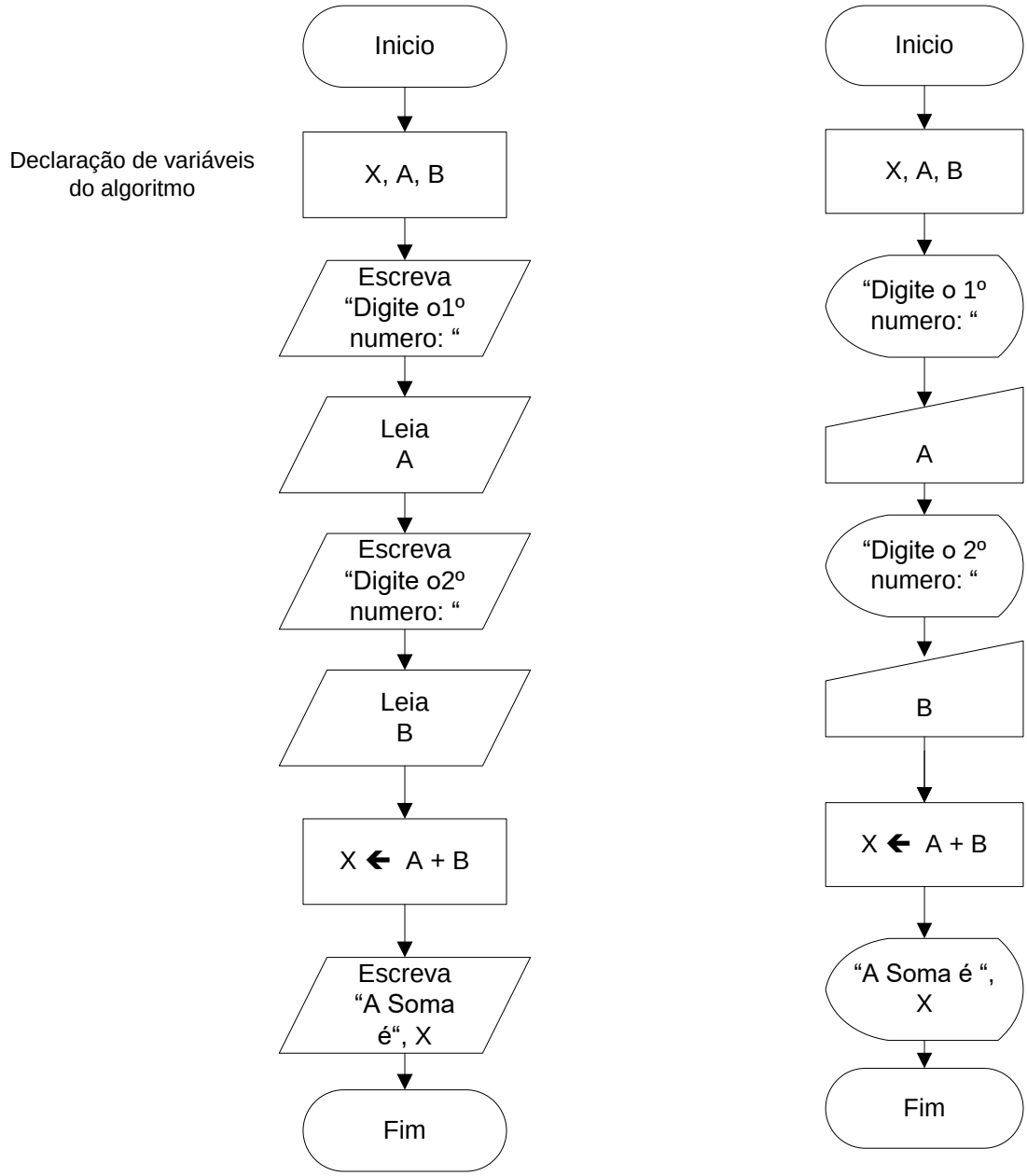
Dividendo → 7 | 3 ← Divisor

MOD → 1 2 ← DIV

Programas Seqüenciais

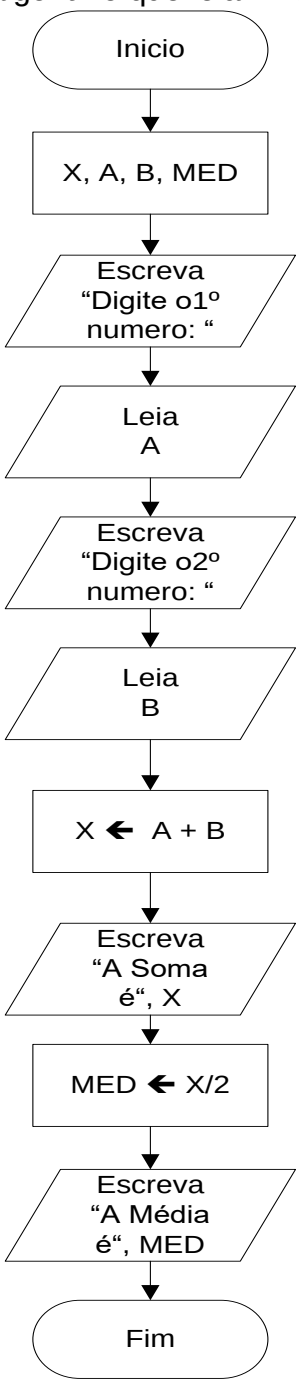
São programas onde são executadas as linhas de instruções uma após a outra, seqüencialmente, sem desvios.

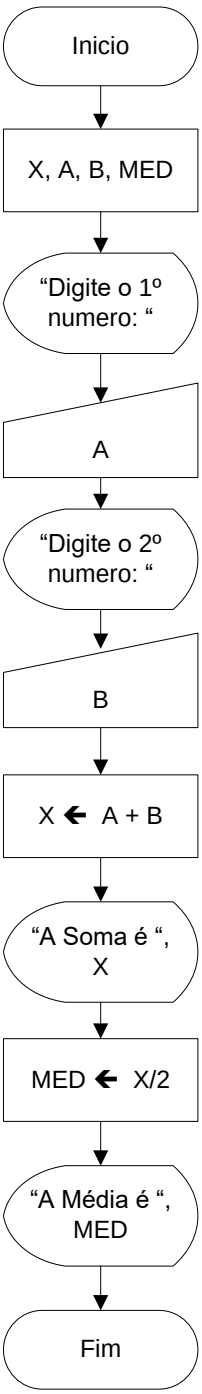
1)Faça um algoritmo para ler dois números inteiros, efetuar e mostrar a SOMA dos mesmos.




```
Algoritmo "SOMA_INTEIROS"
Var
  X, A, B : Inteiro
Inicio
  Escreva ("Digite o 1º número: ")
  Leia (A)
  Escreva ("Digite o 2º número: ")
  Leia (B)
  X  <- A + B  // ou  X := A + B
  Escreval ("A soma é ", X)
Fimalgoritmo
```

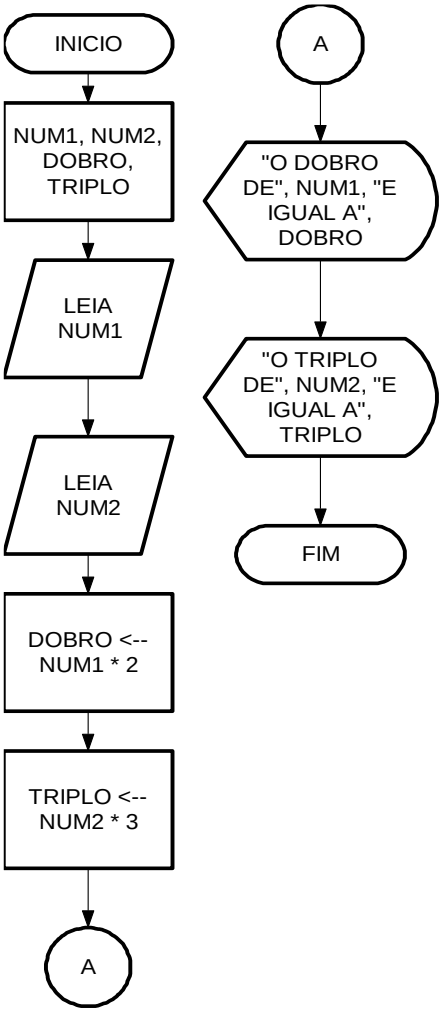
2)Faça um algoritmo que leia 2 números reais, efetue e mostre a SOMA e a MÉDIA:





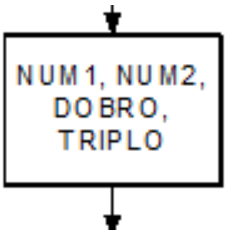

```
Algoritmo "SOMA_MEDIA_REAIS"
Var
    X, A, B, MED : real
Inicio
  Escreva ("Digite o 1º número: ")
  Leia (A)
  Escreva ("Digite o 2º número: ")
  Leia (B)
  X  <- A + B  // X := A + B
  Escreval ("A soma é ",X)
  MED <- X/2
   Escreval ("A média é ",MED)
Fimalgoritmo
```

3) Faça um algoritmo que receba dois números inteiros, calcule e mostre o dobro do primeiro e o triplo do segundo.

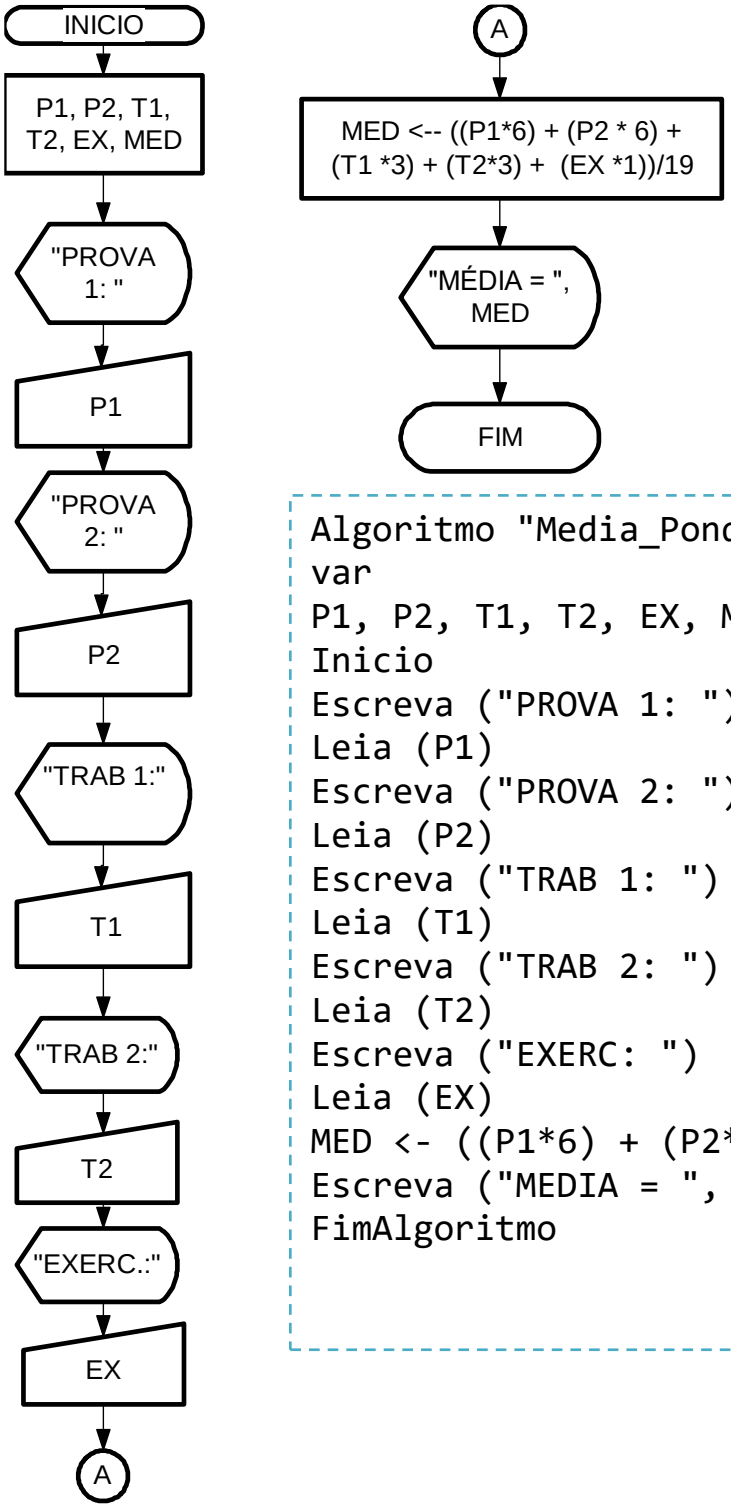



```
Algoritmo "Dobro_triplo"
var
 NUM1, NUM2, DOBRO, TRIPLO: Inteiro
Inicio
Leia (NUM1)
Leia (NUM2)
DOBRO <- NUM1 * 2
TRIPLO <- NUM2 * 3
Escreval ("O DOBRO DE ", NUM1, " E IGUAL A",
DOBRO)
Escreval ("O TRIPLO DE ", NUM2, " E IGUAL A",
TRIPLO)
FimAlgoritmo
```

Nota-se no diagrama acima que foi usada uma representação para declarar as variáveis do algoritmo. Essa prática é recomendada para dar mais clareza ao algoritmo. Veja a seguir a figura (retângulo):



4) Faça um algoritmo que receba pelo teclado as notas de duas provas, dois trabalhos e uma lista de exercícios. Calcule e mostre a média, sabendo que as provas têm peso 6, os trabalhos, peso 3 e a lista, peso 1.




```
Algoritmo "Media_Ponderada"
var
P1, P2, T1, T2, EX, MED: Real
Inicio
Escreva ("PROVA 1: ")
Leia (P1)
Escreva ("PROVA 2: ")
Leia (P2)
Escreva ("TRAB 1: ")
Leia (T1)
Escreva ("TRAB 2: ")
Leia (T2)
Escreva ("EXERC: ")
Leia (EX)
MED <- ((P1*6) + (P2*6)+(T1*3)+(T2*3)+(EX*1))/19
Escreva ("MEDIA = ", MED)
FimAlgoritmo
```

Exercícios - Programas Seqüenciais

1. Fazer o algoritmo que leia dois números inteiros, calcule e exiba a subtração do primeiro pelo segundo.

2. Fazer o algoritmo que leia dois números inteiros, calcule e exiba a subtração do segundo pelo primeiro.

3. Fazer o algoritmo que leia dois números inteiros, calcule e exiba a multiplicação dos dois números.

Reais

4. Fazer o algoritmo que receba duas notas de um aluno e seus respectivos pesos, calcule e exiba a média ponderada dessas notas.

MP = (N1 * P1 + N2 * P2) / (P1 + P2)

5. Fazer o algoritmo que leia o valor de um depósito e o valor da taxa de juros. Calcule e exiba o valor do rendimento e o valor total (depósito + rendimento).

6. Fazer o algoritmo que leia um número inteiro, calcule e exiba:
   a) a raiz quadrada desse número; b) esse número elevado ao quadrado.

7. Fazer o algoritmo que leia o valor do salário de um funcionário e o valor do salário mínimo. Calcule e exiba quantos salário mínimos ganha esse funcionário.

8. Fazer o algoritmo e o programa Pascal que calcule e exiba a área de um triângulo.

9. Fazer o algoritmo que calcule e exiba a área de um quadrado.

10. Fazer o algoritmo que calcule e exiba a área de um círculo.

11. Fazer o algoritmo que calcule e exiba a área de um trapézio.

12. Fazer o algoritmo que calcule e exiba a área de um retângulo.

13. Fazer o algoritmo que calcule e exiba a área de um losango.

14. Fazer o algoritmo que leia o salário de um funcionário, calcule e exiba o valor do imposto de renda a ser pago, sabendo-se que o imposto equivale a 5% do salário.

15. Fazer o algoritmo que leia o salário de um funcionário, calcule e exiba o novo salário, sabendo-se que este sofreu um aumento de 25%.

16. Sabe-se que o quilowatt de energia custa um quinto do salário mínimo. Fazer o algoritmo que leia o valor do salário mínimo e a quantidade de quilowatts gasta por residência. Calcule e exiba:
    a) valor, em reais, de cada quilowatt;
    b) valor, em reais, a ser pago por essa residência;
    c) novo valor a ser pago por essa residência, a partir de um desconto de 15%.

17. Fazer o algoritmo que leia o peso de uma pessoa em quilograma, um valor inteiro, calcule e exiba:
    a) peso dessa pessoa em gramas;
    b) se essa pessoa engordar 15%, qual será seu novo peso em gramas.

18. Fazer o algoritmo que leia a quantidade de litros de uma represa, um valor inteiro, calcule e exiba;
    a) a quantidade total em litros se ocorrer um aumento de 34% na sua capacidade, após um período de chuvas;
    b) a quantidade total em mililitros se ocorrer uma redução de 40% na sua capacidade, após um período de seca.

19. Construa o algoritmo que leia dois nºs inteiros, calcule a soma e mostre o resultado.
20. Construa o algoritmo que leia três nºs inteiros, calcule a soma e mostre o resultado.
21. Construa o algoritmo que leia dois números reais, calcule a média aritmética entre eles e mostre o resultado.
22. Construa o algoritmo que leia três nºs reais, calcule a média aritmética entre eles e mostre o resultado.
23. Construa o algoritmo que leia o preço unitário e a quantidade vendida de um produto. Calcule e mostre o valor total vendido.
24. Construa o algoritmo que calcule e mostre a média aritmética entre quatro notas bimestrais quaisquer fornecidas por um aluno.
25. Construa o algoritmo que leia uma temperatura em graus Centígrados e apresentem-na convertida em graus Fahrenheit. A fórmula de conversão é F = (9 * C + 160) / 5. Onde F é a temperatura em Fahrenheit e C é a temperatura em Centígrados.
26. Construa o algoritmo que leia uma temperatura em graus Fahrenheit e apresentem-na convertida em graus Centígrados. A fórmula de conversão é C = (F – 32) * (5 / 9). Onde F é a temperatura em Fahrenheit e C é a temperatura em Centígrados.
27. Construa o algoritmo que calcule e apresente o valor do volume de uma lata de óleo, utilizando a fórmula: VOLUME = 3.14159 * (RAIO * RAIO) * ALTURA.
28. Construa o algoritmo e o programa Pascal que efetuem o cálculo da quantidade de litros de combustível gastos em uma viagem, utilizando-se um automóvel que faz 12 km por litro. Para obter o cálculo, o usuário deverá fornecer o tempo gasto na viagem e a velocidade média durante a mesma. Desta forma, será possível obter a distância percorrida com a fórmula DISTÂNCIA = TEMPO * VELOCIDADE. Tendo o valor da distância, basta calcular a quantidade de litros de combustível utilizada na viagem com a fórmula: LITROS_USADOS = DISTÂNCIA / 12. O algoritmo e o programa Pascal deverão apresentar os valores da velocidade média, tempo gasto na viagem, a distância percorrida e a quantidade de litros utilizada na viagem.

Bons estudos

Para uma reflexão...sempre Antes dos estudos...

O Bambu Chinês *(Autor desconhecido)*

Depois de plantada a semente do bambu, não se vê nada por aproximadamente 5 anos, exceto um lento desabrochar de um diminuto broto a partir do bulbo.

Durante 5 anos, todo o crescimento é subterrâneo, invisível a olho nu, mas uma maciça e fibrosa estrutura de raiz que se estende vertical e horizontalmente pela terra está sendo construída.

Então, no final do 5º ano, o bambu chinês cresce até atingir a altura de 25 metros.

O escritor Stephen Covey escreveu: "Muitas coisas na vida pessoal e profissional são iguais ao bambu chinês. Você trabalha, investe tempo, esforço, faz tudo o que pode para nutrir seu crescimento, e às vezes não vê nada por semanas, meses ou anos. Mas se tiver paciência para continuar trabalhando, persistindo e nutrindo, o seu 5.º ano chegará, e com ele virão um crescimento e mudanças que você jamais esperava..."

O bambu chinês nos ensina que a ***persistência*** *e a* ***paciência*** *são os caminhos para alcançarmos nossos sonhos e que é* ***preciso muita fibra para chegar às alturas, e ao mesmo tempo, muita humildade e flexibilidade para se curvar ao chão.***

Bons estudos